



PunkBackup — iPhone Setup (WiFi Backup Shortcut)

This manual shows the PC app in **English** — go to "■ **Settings**" → "**English**" if it isn't already switched (instant, no restart).

This guide walks you through creating, in the iPhone's built-in **Shortcuts** app (no extra download needed), the shortcut that sends your new photos and videos to your PC over WiFi.

Before you start, have these ready

This system supports several devices/people (profiles) on the same PC. **Each device needs its own profile and its own token** — so if you're setting up more than one iPhone/iPad, repeat these steps in the PC app for each one.

On your PC: 1. Open the app and click "■ **Start backup**". 2. In "■ **Settings**", click "+ **Add profile**" and name it after THIS device (e.g. iPhone de Laura), then choose its destination folder when prompted. 3. The app shows you (and auto-copies) that profile's **token** — it only works for this one device.

Note down these values:

Value	Where to find it in the app	Example
Server address (same for every profile)	"Address" field	<code>http://[YOUR_PC_NAME].local:8787</code>
Fallback IP (in case the address above doesn't resolve from the iPhone)	"Fallback IP" field	<code>http://192.168.1.50:8787</code>
This profile's token	"Copy token" button on the profile row you just created	a long string of letters/numbers

Your iPhone and your PC must be on the **same WiFi network**. If you add another device later, create its own profile — never reuse another device's token.

Design note: this shortcut does NOT use any "already backed up" album on the iPhone. The decision of what to upload is always made by the server, by checking what actually exists in your profile's destination folder right now — so if you switch USB drives/destination folders on the PC, the system adjusts itself automatically instead of incorrectly "remembering" something that never actually made it to

that folder.

Step 1 — Create the main Shortcut ("PunkBackup")

1. Open the **Shortcuts** app.
2. **My Shortcuts** tab → + button → **Add Action** (new shortcut).
3. Tap the name at the top and rename it to: **PunkBackup**.
4. Add the following actions **in this order** (search each by name using the magnifying glass):

1) Store the server address and token as variables

- **Text** action → type the server address, e.g. `http://[YOUR_PC_NAME].local:8787`
- **Set Variable** action → name it **ServerURL** → value: the Text above.
- **Text** action → paste the Token **for this profile/device** you copied from the app.
- **Set Variable** action → name it **Token** → value: the Text above.

These are the only two actions you'll ever need to edit again — for example if you regenerate this profile's token from the app ("Regenerate token" button).

2) Tell the server a backup run is starting

- **Get Contents of URL** action:
- URL: **ServerURL** + `/run/start` (tap the **ServerURL** variable, then type `/run/start` right after it).
- Method: **POST**
- Headers: add one → Key **X-Backup-Token** → Value: the **Token** variable.
- **Get Dictionary Value** action → Key: **run_id** → (applied to the result of the previous step).
- **Set Variable** action → name it **RunID** → value: the result above.

3) Find ALL photos/videos

- **Find Photos** action (may be labeled "Filter Photos" on some iOS versions):
- No album filter — we want **every** photo/video; the server decides which ones it already has.
- Sort by: **Date Taken, Oldest First**.
- (Optional) Limit: if you have a huge library (several thousand), set a limit (e.g. 300) to keep each run shorter — since the check is lightweight and the system is incremental, you can just run the shortcut again and it keeps making progress.

4) Loop through each item: ask first, only upload if missing

- **Repeat with Each** action over the result of the previous step. Inside the "Repeat" block:
 - a. **Format Date** action: - Date: tap "Repeat Item" and choose the **Date Taken** attribute. - Format: **Custom** → type: `yyyy-MM-dd'T'HH:mm:ss` - Store this as a variable named **TakenAt** (**Set Variable** action).

a.2. **Text** action — build the full filename, **with extension** (■■ important: Shortcuts' "File Name" attribute on its own does **not** include the extension — skip this step and you'll end up with files like `IMG_1234` instead of `IMG_1234.HEIC`): - In the text field, insert: **Repeat Item** → choose the **File Name** attribute → type a period `.` (no spaces) → insert **Repeat Item** again → choose the **File Extension** attribute. - It should read something like: `[File Name].[File Extension]` - Store this as a variable named `FileName` (**Set Variable** action).

b. **Get Contents of URL** action — the lightweight check, **WITHOUT** the file: - URL: `ServerURL + /check` - Method: **POST** - Request Body: **Form** - Form fields: - filename → value: the **FileName** variable (from step a.2 — NOT the bare "File Name" attribute, which is missing the extension). - taken_at → value: `TakenAt` variable. > No need to send the file size — Shortcuts has no reliable way to give > a plain byte count (it always formats it as something like "1.2 MB"), > so the check only uses filename + date. The exact content check > happens later, in `/upload`. - Headers: `X-Backup-Token` → `Token` variable. - **Get Dictionary Value** action → key missing → applied to that result.

c. **If** action: condition = the value above **has any value** (this is how the server tells you "it's missing, upload it"; when a file is already backed up, the server simply omits the `missing` field entirely, so this condition is automatically false and the "If" is skipped — you do NOT need "is equal to" or to type `false` anywhere, "has any value" is already the right option):

- ****Get Contents of URL**** (inside the "If"):
 - URL: tap the field and insert, **IN THIS ORDER**, inside the same text field: the ****ServerURL**** chip → type `/upload?filename=` → insert the ****FileName**** variable chip (the same one from step a.2 — NOT the bare "File Name" attribute, which is missing the extension) → type `&taken_at=` → the ****TakenAt**** chip → type `&run_id=` → the ****RunID**** chip.
 - > Inserting the chips directly into the URL field (instead of > building the text separately with "Combine Text") makes > Shortcuts URL-encode them automatically.
 - Method: ****POST****
 - ****Request Body**** → change it to ****File**** (no longer Form).
 - In the value field that appears, open the variable bar and insert ****Repeat Item**** — ****once, and don't tap the chip again afterward****. (Tapping it again opens a "Name/Album/Width..." property menu — if that happens and you end up with anything other than plain "Repeat Item", clear it with "Clear Variable" and re-insert it without touching it a second time.)
 - Headers: `X-Backup-Token` → `Token` variable.
- (Nothing needed in the "Otherwise" branch — if it was already backed up, just move on to the next item.)

5) Close out the run and show you the result

- **Get Contents of URL** action:
- URL: `ServerURL + /run/finish`
- Method: **POST**, Form with field `run_id` = `RunID` variable.

- Header X-Backup-Token → Token variable.
- **Get Dictionary Value** actions (one per field: `files_new`, `files_skipped`, `files_conflict`) applied to that result.
- **Text** action to build a message, e.g.: Backup complete ■\nNew: `[files_new]`\nAlready had: `[files_skipped]`\nConflicts: `[files_conflict]`
- **Show Notification** (or **Show Result**) action with that text.

Save the shortcut — you can now tap it manually to test it.

On how long each run takes: the first time you run this against a large library, it will check EVERY photo (the `/check` call is lightweight — just filename/size/date, no file transfer — so it's fast even for thousands of photos). Later runs cover the same full library but feel much faster in practice, since most items will already be backed up and `/check` answers instantly for those.

Step 2 — Short "check status" shortcut

Optional but recommended — lets you check status without running a full backup.

1. Create a new shortcut named `Backup Status`.
2. Repeat the two "Text" + "Set Variable" actions from Step 1.1 (`ServerURL`, `Token`).
3. **Get Contents of URL** action: URL = `ServerURL + /status`, Method **GET**, header X-Backup-Token → `Token`.
4. **Get Dictionary Value** actions for `last_backup_at` and `total_files_backed_up`.
5. **Show Notification** action with those values.

Step 3 — Automate it: run on its own when you get home

1. Open **Shortcuts** → **Automation** tab → + → **Create Personal Automation**.
2. Choose **Wi-Fi** → select your home network: `[YOUR_WIFI_NETWORK]`.
3. Leave **Connects** checked.
4. Tap **Next** → **Add Action** → search for and choose your PunkBackup shortcut.
5. Tap **Next** → **Done**.
6. Important: turn off **"Ask Before Running"** for this automation. The first time it runs it may ask for a one-time security confirmation — accept it. After that it will run silently every time you join that WiFi.

You can also run PunkBackup manually anytime by tapping it in the Shortcuts app, or saying "Hey Siri, PunkBackup".

Notes

- The system **never deletes or modifies anything on your iPhone** — it only reads photos and uploads them.
- The truth about "what's already backed up" lives **only in the PC's destination folder**, never on the iPhone. If you switch your profile's folder/USB drive on the PC, the shortcut automatically re-sends whatever is missing from the new folder — you don't need to do anything different on the iPhone.
- Exact action names can vary slightly between iOS versions; if you can't find an action with the exact name above, search by a keyword instead (e.g. "dictionary", "URL", "repeat").